

Macrosystems EDDIE: Climate change effects on lake temperatures

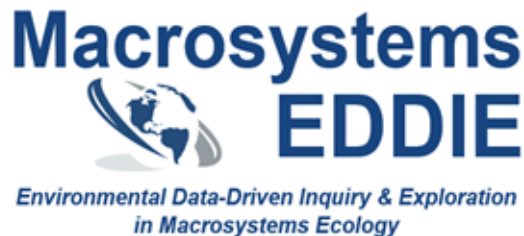
Carey, C.C., S. Aditya, K. Subratie, R. Figueiredo, and K.J. Farrell. 16 Oct 2024.

Macrosystems EDDIE: Climate Change Effects on Lake Temperatures.

Macrosystems EDDIE Module 1, Version 2.

<http://module1.macrosystemseddie.org>

Module development supported by NSF DEB 1245707, ACI 1234983, & EF 1702506



Overview of today

- Short overview of lake modeling and some background on the tools we will be working with
- **Activity A:** Plot water temperatures from a lake model.
- **Activity B:** Develop a climate scenario, generate hypotheses, and model how your lake responds.
- **Activity C:** Create a custom coding function and use a for-loop to run many lake model simulations.

Lakes are changing worldwide...

- In response to altered climate, some regions are experiencing different temperature, precipitation, and wind conditions than in the past.
- Because of their position in the landscape and their thermal inertia, lakes can be powerful indicators of climate change.



Image: Wikimedia commons



Image: Wikimedia commons

Our focal question:

How will lake thermal structure
change in response to
altered climate?

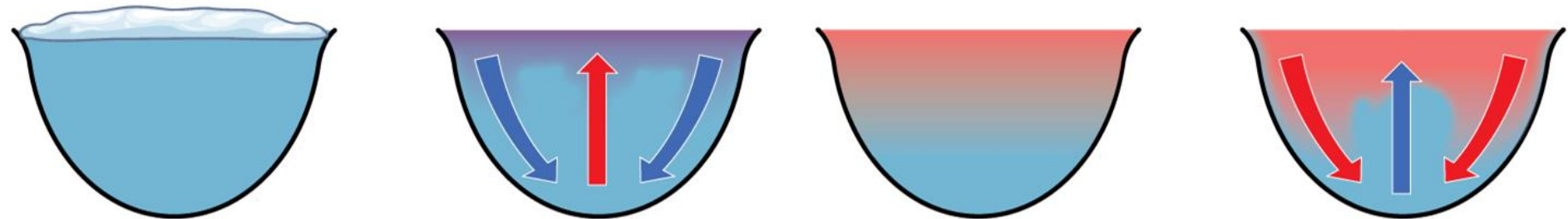
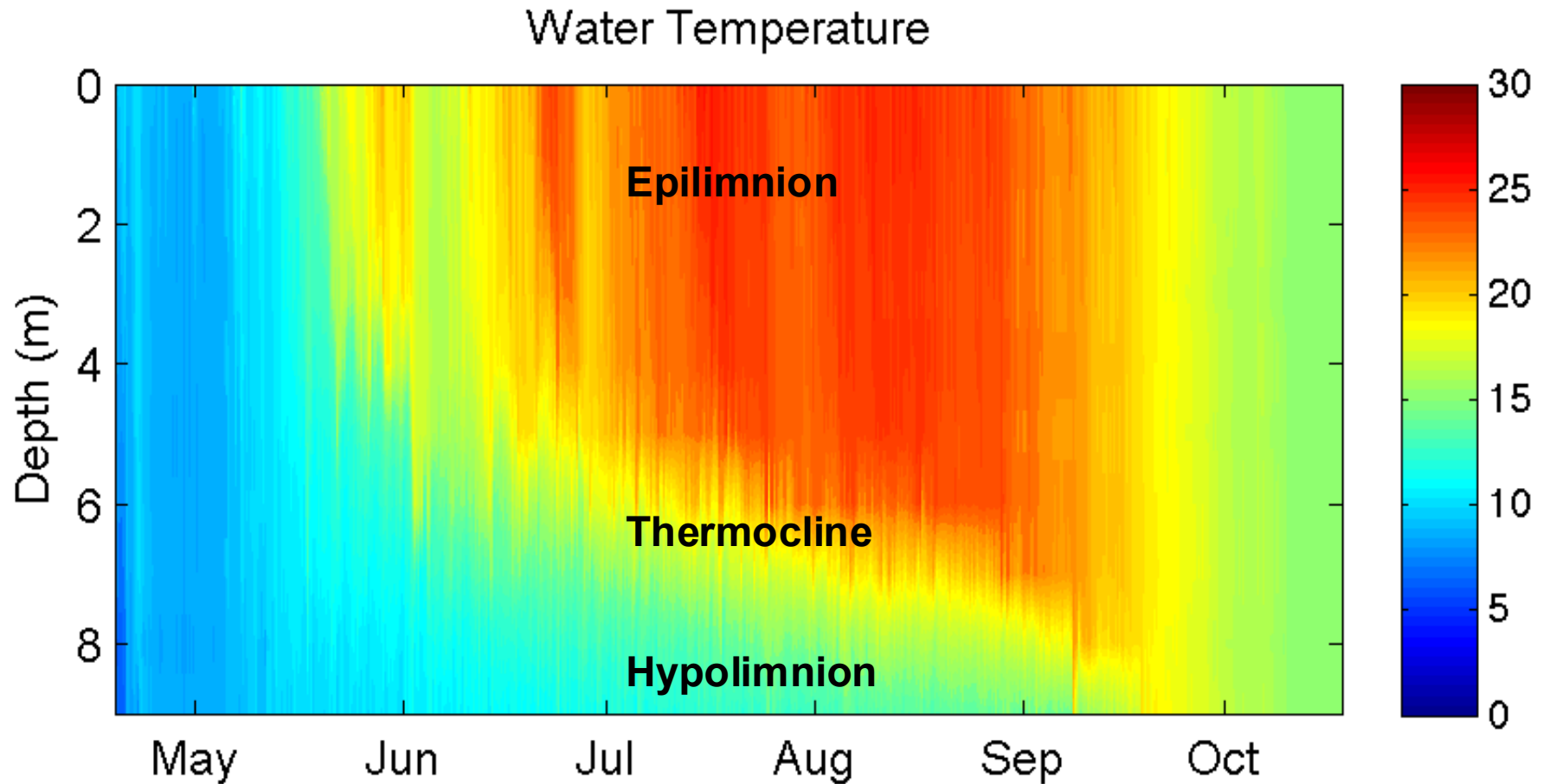


Image modified from: Figure 44.10 in "Ecology and the Biosphere" (Candela Learning)

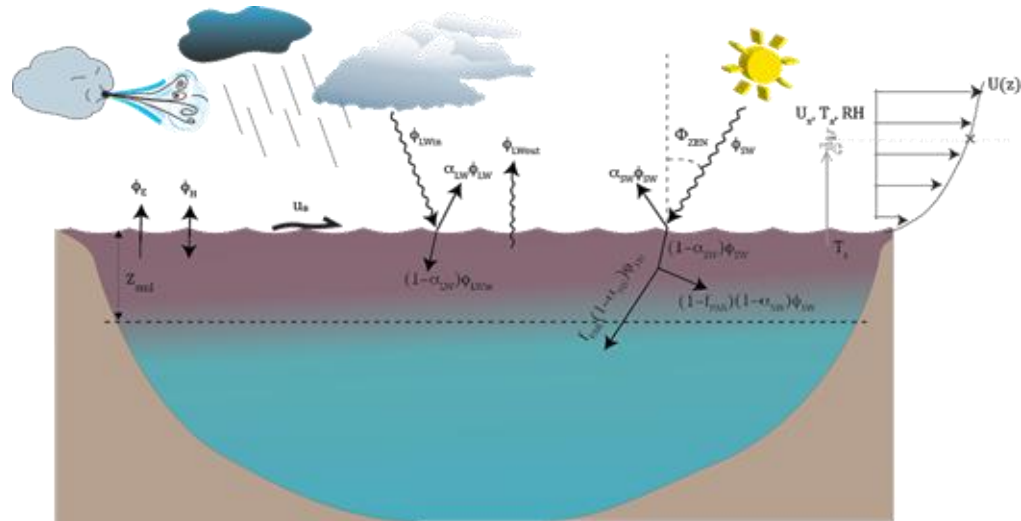
What is lake thermal structure?



Lake Sunapee, NH, USA: data from Lake Sunapee Protective Association buoy, 2012

What are the drivers of thermal structure?

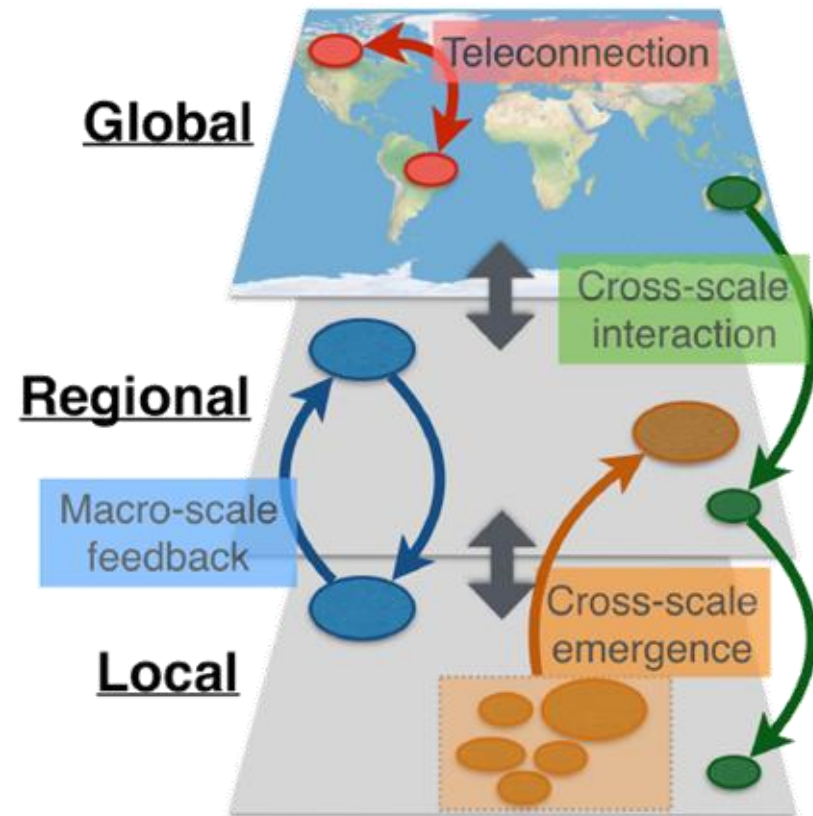
- Solar radiation
- Air temperature
- Wind
- Precipitation
- Inflow & outflow streams



- All of these factors will interact to control water temperature, thermocline depth, mixing, and more!

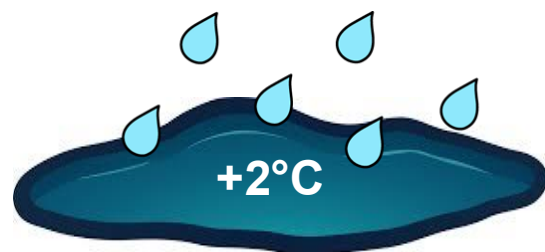
Using a macrosystems ecology approach to study lake thermal structure

- Drivers of lake thermal structure occur at both local and regional scales
 - Local = e.g., inflow streams
 - Regional = e.g., air temperature
- Macrosystems ecologists study ecological dynamics and feedbacks at multiple interacting spatial and temporal scales
- We can study local and regional drivers using high-frequency sensor data + simulation models



Models to understand multiple drivers

- How can we test the effects of drivers from different scales on lake thermal structure?
 - Impossible to experimentally manipulate all possible drivers in a lake!
- Simulation modeling allows us to explore what would happen if we changed one driver, or multiple drivers at once
 - For example, how would lake thermal structure change if air temperatures were 2°C warmer than they are now? 4°C? 6°C?
 - What if rain also increased 50% on those lakes?



GLM: General Lake Model

- **Authors:** Matt Hipsey, Louise Bruce, and David Hamilton



- The **General Lake Model** (GLM) is an open-access model for simulating lake dynamics. It simulates vertical stratification and mixing and accounts for the effect of inflows/outflows, surface heating and cooling.
- GLM has been designed to be an open-source community model developed in collaboration with members of the **Global Lake Ecological Observatory Network (GLEON)** to integrate with lake sensor data.
- Available from: <http://aed.see.uwa.edu.au/research/models/GLM/>

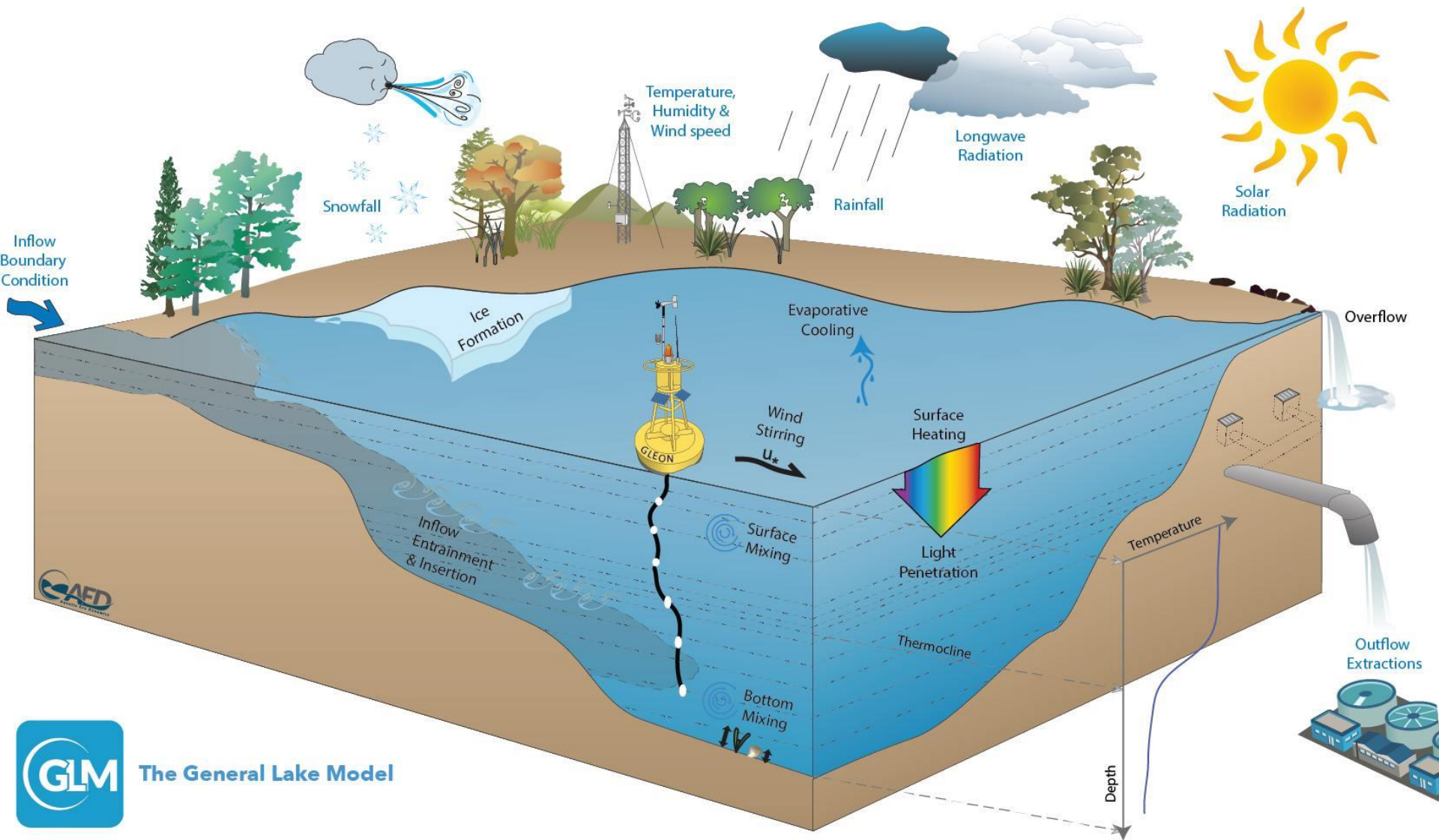


Figure: Hipsey et al. 2014

Basic structure of the model

- You need to create a new folder (directory) on your computer.
- Within this folder, you will have:
 - 1) A high-frequency meteorological CSV file ('met file') that forces the model
 - e.g., "met_hourly.csv"
 - 2) An 'nml' text file that acts as a 'master' script
 - glm2.nml
 - 3) High-frequency Inflow/outflow CSV files that specify the temperature and flow rate of connected streams
 - Today we're studying a closed lake without inflows or outflows



Example met file

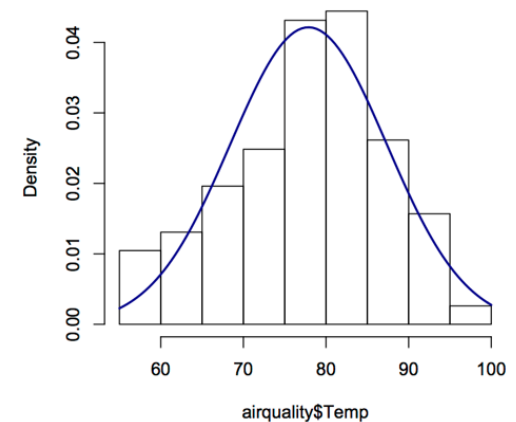
	A	B	C	D	E	F	G
1	time	ShortWave	LongWave	AirTemp	RelHum	WindSpeed	Rain
2	2009-01-01 00:00	0	237.0800018	-12.32001343	71.3469252	11.25574053	0
3	2009-01-01 01:00	0	237.0800018	-13.0000061	70.61931244	11.61440932	0
4	2009-01-01 02:00	0	237.0700073	-13.66998901	69.5395646	11.9717879	0
5	2009-01-01 03:00	0	212.8999939	-14.35001221	68.17292338	12.33103782	0
6	2009-01-01 04:00	0	212.8899994	-14.85999146	68.26885627	12.45128543	0
7	2009-01-01 05:00	0	212.8899994	-15.37000122	68.22720667	12.58038186	0
8	2009-01-01 06:00	0	178.0099945	-15.88001099	68.07701147	12.71270623	0
9	2009-01-01 07:00	0	178.0099945	-15.82999268	68.560877	12.30275177	0
10	2009-01-01 08:00	0	178.0099945	-15.78000488	69.0241756	11.90672061	0
11	2009-01-01 09:00	0	175.9700012	-15.72998657	69.46721996	11.50951766	0
12	2009-01-01 10:00	0	175.9700012	-15.91000977	70.3181745	11.37918273	0
13	2009-01-01 11:00	0	175.9600067	-16.10001221	71.21325022	11.24893784	0
14	2009-01-01 12:00	0	154.0700073	-16.28000488	72.06085374	11.12530937	0
15	2009-01-01 13:00	77.08799744	154.0700073	-16.13999023	71.84831464	11.38470936	0
16	2009-01-01 14:00	196.1999969	154.0700073	-16.0000061	71.65033449	11.65082869	0
17	2009-01-01 15:00	317.9200134	146.8600006	-15.85001221	71.39024308	11.91713029	0
18	2009-01-01 16:00	391.9039917	146.8600006	-15.13001099	72.36215735	11.65626426	0
19	2009-01-01 17:00	429.428009	146.8699951	-14.41000977	73.02151614	11.40773869	0
20	2009-01-01 18:00	371.394989	153.4600067	-13.69000854	73.38292443	11.16527203	0

Example .nml file

```
glm2.nml
90 !-----
91 ! lake details
92 !-----
93 !
94 ! name          [string]
95 !               name of the lake
96 ! latitude      [float, minimum = -90, maximum = 90, unit = deg North]
97 !               latitude
98 ! longitude      [float, minimum = -360, maximum = 360, unit = deg East]
99 !               longitude
100 ! base_elev     [float]
101 !               base elevation (m)
102 ! crest_elev    [float]
103 !               crest elevation (m)
104 ! bsn_len        [float]
105 !               basin length at crest (m)
106 ! bsn_wid        [float]
107 !               basin width at crest (m)
108 ! bsn_vals       [integer]
109 !               number of depth points on height-area relationship
110 ! H              [float]
111 !               elevations (m) (comma separated list, len=bsn_vals)
112 ! A              [float]
113 !               area (m2) (comma separated list, len=bsn_vals)
114 !
115 !-----
116 &morphometry
117   lake_name      = 'Mendota'
118   latitude       = 43
119   longitude      = -89.41
120   bsn_len        = 9500
121   bsn_wid        = 7400
122   bsn_vals       = 15
123   !   H(m)   A(m2 * 1000) V(m3 * 1000)
124   H = 375.00640,376.79166,378.57691,380.36217,382.14743,383.93269, 385.71794,387.50320,389.28846,391.07979,392.87104,394.65229,396.43354,398.21479,400.00000
125   A = 0.00000,2827226.39,5654452.79,8481679.18,11308905.58,14136131.97,16963358.37,19790584.76,22617818.15,25528184.44,2827226.39,5654452.79,8481679.18,11308905.58,14136131.97,16963358.37,19790584.76,22617818.15,25528184.44
```

We will run GLM using R

- R is a statistical environment that can run on different computer operating systems (PC, Mac, Linux)
- R is reproducible, free(!), and easy to download
- R can run stats, make figures, and do a suite of different analyses in many disciplines
- Many packages for R to merge with other tools (including GLM!)
- R is the programming language of choice for most ecologists



GLM-associated R packages

- **GLMr**: holds the current version of the GLM model and allows you to run GLM on any operating system
- **glmtools**: has different functions for analyzing and plotting GLM output from GLMr
- Developed by Jordan Read and Luke Winslow
- Available from: <https://github.com/GLEON> & <https://github.com/USGS-R/glmtools>



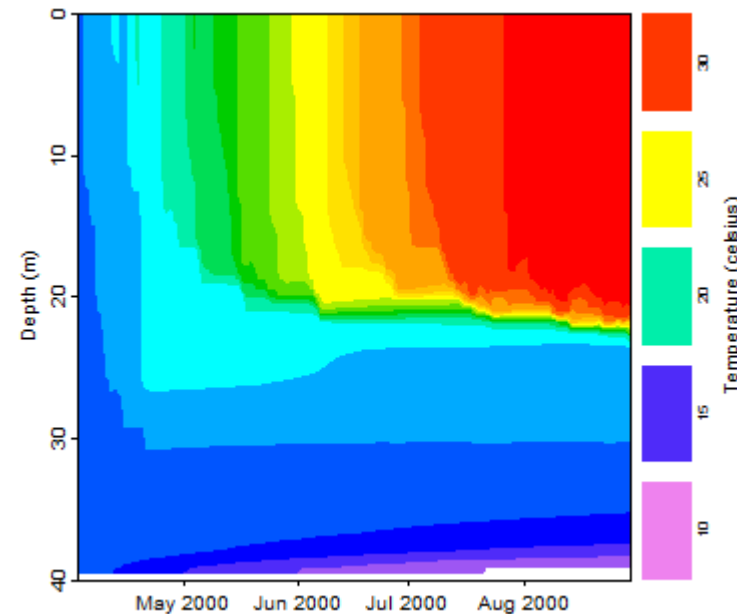
Learning objectives of today's module

- Set up and run the General Lake Model (GLM) in the R statistical environment to simulate lake thermal structure
- Understand how GLM configuration files, high-frequency driver data, and output files are organized and used
- Modify the input meteorological data for one GLM model to simulate the effects of different climate scenarios on lake thermal structure
- Interpret model output from GLM simulations to understand how changing climate will alter lake thermal characteristics
- Create a custom coding function and use a for-loop to set up and run many model simulations with varying input meteorological data
- Explore the application of R functions and for-loops for modeling climate change effects on lakes

Activity A

With a partner (work in pairs):

- 1) Download the zipped folder containing the R Markdown (.Rmd) file and other files to run this module. Extract to your Desktop
- 2) Download GLM files and R packages onto your computer
- 3) Run the model and look at the thermal output
- 4) Examine how your model output compares to the observed field data for your lake

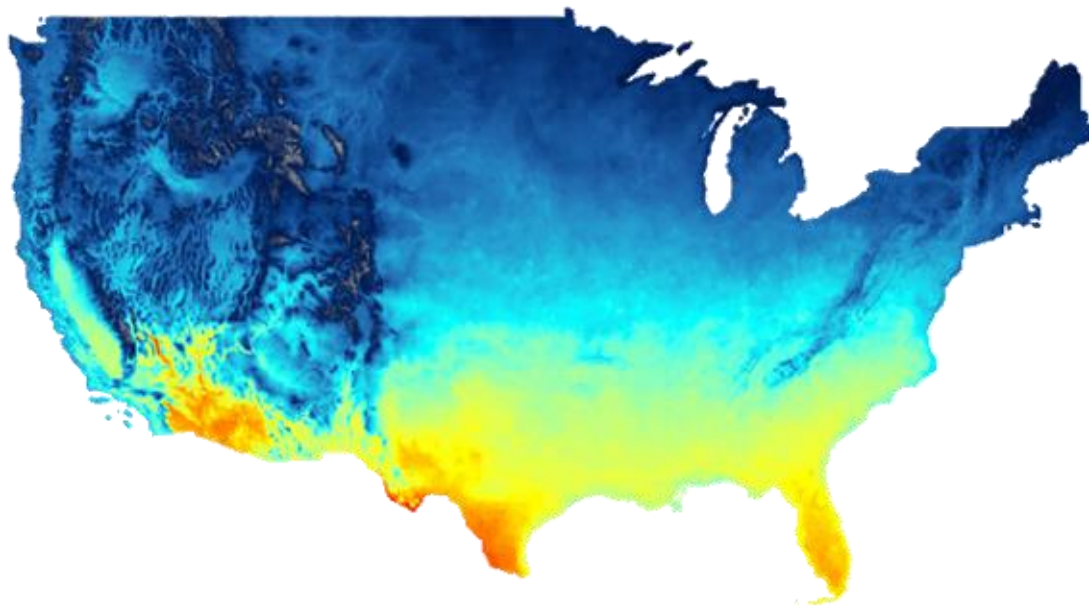


Activity B

- Develop a climate scenario for any region and explore how it will affect lake thermal structure
 - **Develop a hypothesis** about how climate change (e.g., altered temperature) may alter lake thermal structure
 - Create a corresponding meteorological input file
 - Run your model and examine the lake's response:
 - How does your scenario alter the temperature profile in your lake over time?
 - How does the thermocline depth change?
 - How does the timing of stratification and evaporation change?
 - Compare the modeled output to the observed data
 - **Create a few figures to highlight your climate scenario and share with the class!**

Need ideas for a climate scenario?

- Check out climatedata.us
 - Temperature & precipitation predictions for US across the year under different emission scenarios
 - For global projections, try: climatedata.us/global-projections



Converting units?

As you edit the met_hourly file to create your climate scenario, keep the following units in mind!

Variable	Unit	Example	Conversion
AirTemp	°C	20.05	
WindSpeed	meters · sec ⁻¹	1.87	To convert from miles per hour, multiply by 0.447
Rain	meters · day ⁻¹	0.000592	To convert from inches per day, multiply by 0.0254

Activity C

- How do you run many lake simulations using different climate scenarios? Is there a more efficient way to run and analyze lots of different simulations rather than running the code manually?
- YES! by using R functions and for-loops
- A **function** contains code that does a specific task
 - You can create a function to perform any task you like in R!
- A **for-loop** runs a section of code repeatedly
 - You can start the for-loop and let your computer run many lake simulations while you.... Have a snack! Take a nap! (Etc.)

Custom functions in R

- First, define the function

Give your function a descriptive name that is relevant to the task it performs

The inputs you provide to your function are called “arguments”

```
my.function.name <- function(argument1, argument2){  
  sum = argument1 + argument2  
  return(sum)  
}
```

'return' indicates that we want 'sum' to be the output of the function

The code in the curly brackets {} is run every time the function is called; a function can be as simple or complex as you like!

Custom functions in R

- Then you can call the function!

1. Type the name of your function, followed by parentheses ()

2. Supply the arguments
(argument1 = 7, argument2 = 12)

```
> my.function.name(7,12)  
[1] 19
```

3. When you run the function, you will get an output ('sum')

Custom functions to create climate change scenarios

- In Activity C, you will complete a demonstration of how a custom function can be used to create many lake climate change scenarios
- Then, you will have the opportunity to build a custom function to create your own set of scenarios!

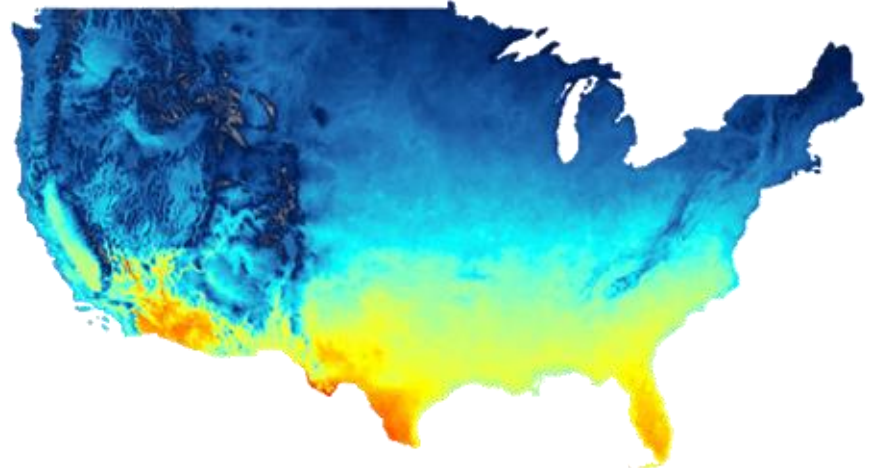


Image: climatedata.us



Images: Wikimedia commons

For-loops in R

- For-loops run a section of code repeatedly

This line specifies how many times to repeat the code (start at 1 and go to 8 repetitions)

All the code in curly brackets `{ }` will be run for every iteration of the for-loop

Here is the for-loop output: we have printed “hello world” eight times!

```
```{r for-loop demo part 1}
for(i in 1:8){
 print("hello world")
}
```
```

[illegible]

For-loops in R

- Within a for-loop, **indexing** allows you to refer to a particular iteration, or individual code run, of the loop.

In this for-loop, the index is specified using the letter 'i'. 'i' is a commonly used symbol for indexing in a for-loop in R

Here is the for-loop output: we have printed the value of 'i' eight times! Notice how the value of 'i' changes each time the for-loop runs

```
```\r for-loop demo part 2}  
for(i in 1:8){
 print(i)
}
```\r
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6  
[1] 7  
[1] 8
```

For-loops to run many lake model simulations

- In Activity C, you will use a for-loop to run many lake model simulations using different climate change scenarios

Every iteration of the for-loop will run a different lake model simulation with a different climate scenario

The output of the for-loop will be many lake model simulations, allowing you to compare among climate change scenarios!

```
```{r for-loop demo part 2}  
for(i in 1:8){
 Code to run lake model
}
```
```

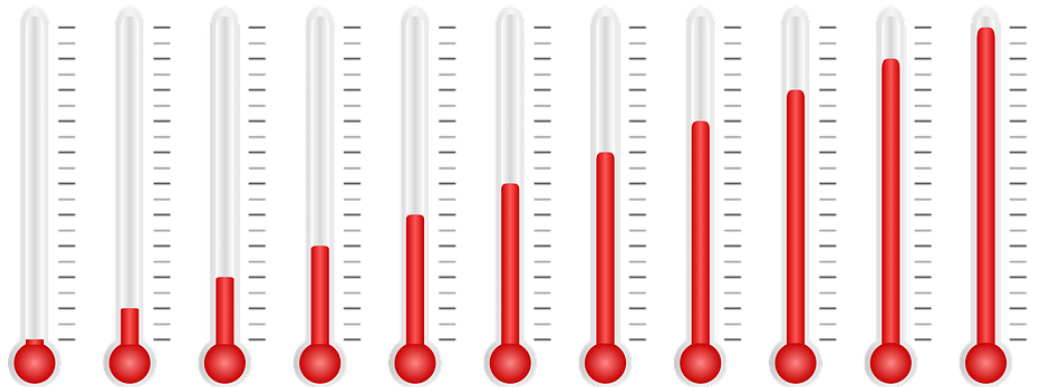


Output for many lake model simulations!

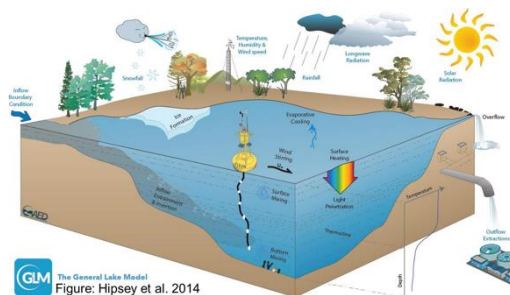


Activity C

- Go through the demo in the R Markdown file which uses a custom function and a for-loop to run multiple air temperature scenarios
- Once you have finished the demonstration, design your own simulation "experiment" with your partner to examine the effects of a set of climate change scenarios of your choice.
- **Create some figures from your simulation and share them with the class.**



Thank you for participating!



GLM The General Lake Model
Figure: Hipsey et al. 2014

